Python 3.12 Concurrency • INSTRUCTOR ANSWER KEY

Course: Python 3.12 • College: VIT • Student Name: _____

Q1. In Python 3.12, how does asyncio manage concurrent I/O operations without spawning OS threads?

[CORRECT] A) Using a single-threaded cooperative event loop that yields control at 'await' expressions

B) By running each coroutine on a separate physical CPU hardware core simultaneously

C) By disabling the Global Interpreter Lock (GIL) automatically for all functions

D) By converting all asynchronous Python functions into synchronous C-extensions at runtime

Key: A (Grounded in Slide 1) — asyncio relies on an event loop running in a single thread, cooperatively pausing and resuming coroutines when I/O is pending.

Distractor Analysis (Common Mistakes):

- **Option B:** Common mistake: Confusing cooperative asynchronous event loops with multi-core parallelism (multiprocessing).
- **Option C:** Common mistake: Assuming asyncio eliminates the GIL; standard CPython coroutines still operate under the GIL.
- **Option D:** Common mistake: Believing asyncio compiles Python to C extensions; it is pure Python bytecode evaluated by the interpreter.

Q2. Which Python interpreter flag treats PendingDeprecationWarning and FutureWarning as fatal runtime exceptions?

[CORRECT] `A) -W error::DeprecationWarning -W error::FutureWarning`

B) --ignore-deprecation-errors

C) -O2 --disable-warnings

D) -B --no-exceptions

Key: A (Grounded in Slide 1) — The -W error flag elevates specified warning categories to fatal exceptions, preventing obsolete APIs from running undetected.

Distractor Analysis (Common Mistakes):

- **Option B:** Common mistake: Inventing flags; CPython uses the -W flag family for warning management.
- **Option C:** Common mistake: -O2 removes docstrings and assert statements, but does not configure warning handling.
- **Option D:** Common mistake: -B prevents writing .pyc files and has nothing to do with warning escalation.

Q3. What is the exact stdout produced by running this verified Python snippet?

```
nums = [1, 2, 3, 4]
res = [x * 2 for x in nums if x % 2 == 0]
print(res)
```

A) [2, 4, 6, 8]

[CORRECT] B) [4, 8]

C) [2, 6]

D) [4, 6, 8]

Key: B (Grounded in Slide 1) — Executing this code under pinned Python runtime yields verified output: '[4, 8]'.

Distractor Analysis (Common Mistakes):

- **Option A:** Common mistake: Forgetting the comprehension filter 'if x % 2 == 0' and doubling all elements.
- **Option C:** Common mistake: Inverting the modulo predicate to filter odd numbers instead of even numbers.
- **Option D:** Common mistake: Off-by-one mistake in modulo arithmetic.

Q4. What is the exact stdout produced by running this verified Python snippet?

```
data = {'a': 10, 'b': 20}
val = data.get('c', 30) + data['a']
print(val)
```

A) 10

[CORRECT] B) 40

C) 30

D) KeyError: 'c'

Key: B (Grounded in Slide 1) — Executing this code under pinned Python runtime yields verified output: '40'.

Distractor Analysis (Common Mistakes):

- **Option A:** Common mistake: Assuming data.get('c', 30) returns None when key 'c' is missing, ignoring default value 30.
- **Option C:** Common mistake: Omitting data['a'] from the addition.
- **Option D:** Common mistake: Confusing dictionary bracket access data['c'] with safe dictionary method data.get('c', 30).

Q5. In the hands-on lab exercise on Slide 1, what critical edge case must the implementation handle to pass all test assertions?

[CORRECT] A) Empty or null collections must return a neutral default value (e.g. 0) without throwing exceptions

B) The function must always assume at least 10 positive integers are present

C) The method must catch and swallow all exceptions silently without returning any value

D) The function must sort the input collection in-place regardless of immutability

Key: A (Grounded in Slide 1) — Robust production code and unit test specifications require guarded handling of empty and null inputs to ensure zero runtime exceptions.

Distractor Analysis (Common Mistakes):

- **Option B:** Common mistake: Assuming non-empty inputs causes IndexOutOfBoundsException or NoSuchElementException on boundary tests.
- **Option C:** Common mistake: Swallowing exceptions hides logic failures and violates the return type contract.
- **Option D:** Common mistake: Mutating input parameters in-place violates immutability and causes unexpected caller side-effects.

Q6. During automated verification of the exercise on Slide 1, why does the naive solution fail the assertion check?

[CORRECT] A) It computes an unchecked aggregate (e.g. total length or only first elements) without validating required element constraints

B) It fails to compile because modern compilers reject the 'return' keyword inside loops

C) It exhausts all system RAM by spawning 1,000,000 OS background processes

D) It triggers a hardware floating-point division-by-zero error on string literals

Key: A (Grounded in Slide 1) — The naive solution represents a common student mistake: taking a shortcut (like returning array length or first elements) instead of performing the required conditional evaluation.

Distractor Analysis (Common Mistakes):

- **Option B:** Common mistake: Compilers fully support return statements inside loops; the failure is logical, not syntactic.
- **Option C:** Common mistake: Confusing basic algorithmic errors with process exhaustion attacks.
- **Option D:** Common mistake: String literals do not undergo floating point arithmetic division.

Q7. What is the exact stdout produced by running this verified Python snippet?

```
vals = [10, 20, 30]
ans = sum(v // 10 for v in vals)
print(ans)
```

A) 60

[CORRECT] B) 6

C) 3

D) 6.0

Key: B (Grounded in Slide 1) — Executing this code under pinned Python runtime yields verified output: '6'.

Distractor Analysis (Common Mistakes):

- **Option A:** Common mistake: Summing original values (10+20+30 = 60) without applying integer division // 10.
- **Option C:** Common mistake: Counting list elements (len(vals) = 3) instead of summing the divided integers.
- **Option D:** Common mistake: Assuming integer division // produces a float; integer division in Python yields an int.

Q8. As emphasized in the summary on Slide 1, why must automated verification normalize memory addresses and timestamps?

[CORRECT] A) To ensure deterministic test assertions by replacing non-deterministic pointer addresses (e.g. 0x7ffe...) and timestamps with stable placeholders

B) To compress the source code so it can fit on older floppy disks

C) Because the operating system forbids printing hexadecimal numbers to stdout

D) To convert all terminal output into uppercase text automatically

Key: A (Grounded in Slide 1) — Deterministic normalization strips transient memory pointers and dynamic system clocks, ensuring test assertion checks never suffer from flaky mismatches.

Distractor Analysis (Common Mistakes):

- **Option B:** Common mistake: Normalization is for test reproducibility, not archaic media storage compression.
- **Option C:** Common mistake: Operating systems routinely print hex pointers; normalization is a testing harness requirement.
- **Option D:** Common mistake: Normalization preserves character casing and semantic structure.